

ハーネスエンジニアリングを やりすぎた話

そのハーネスは解体された

2026-04-24 Harness Engineering Meetup Tokyo #1

Gota(@gota_bar)

自己紹介

Gota (@gotabar)



所属

Agentic AI エンジニア & アナリティクスエンジニア

やってること

AI エージェント開発 / 小売向けデータプロダクト / データ整備

好きなエージェント

pi ... 超シンプルなTerminal型エージェント. システムプロンプトを人間が容易に読める

 **cc-sdd** Public

Turn approved specs into long-running autonomous implementation. A minimal, adaptable SDD harness with Agent Skills for Claude Code, Codex, Cursor, Copilot, Windsurf, OpenCode, Gemini CLI, and Anti...

● TypeScript ☆ 3.2k 🍷 241

cc-sdd v3.0 (2026/4 update)

- 境界をファーストクラス化
- 長時間の自律実装：specのTDD実装 ↔ レビューのClosed Loop
- Slash commandをAgent Skillsに変更

作ってたエージェントハーネス

やりたかったのは、仕様を承認すればあとは放置でその通りに動くソフトウェア開発がEnd-to-Endでできること。惰眠を貪りたい...



超長時間自律: 寝ている間に走らせ、朝には大規模開発が仕様通り完了する



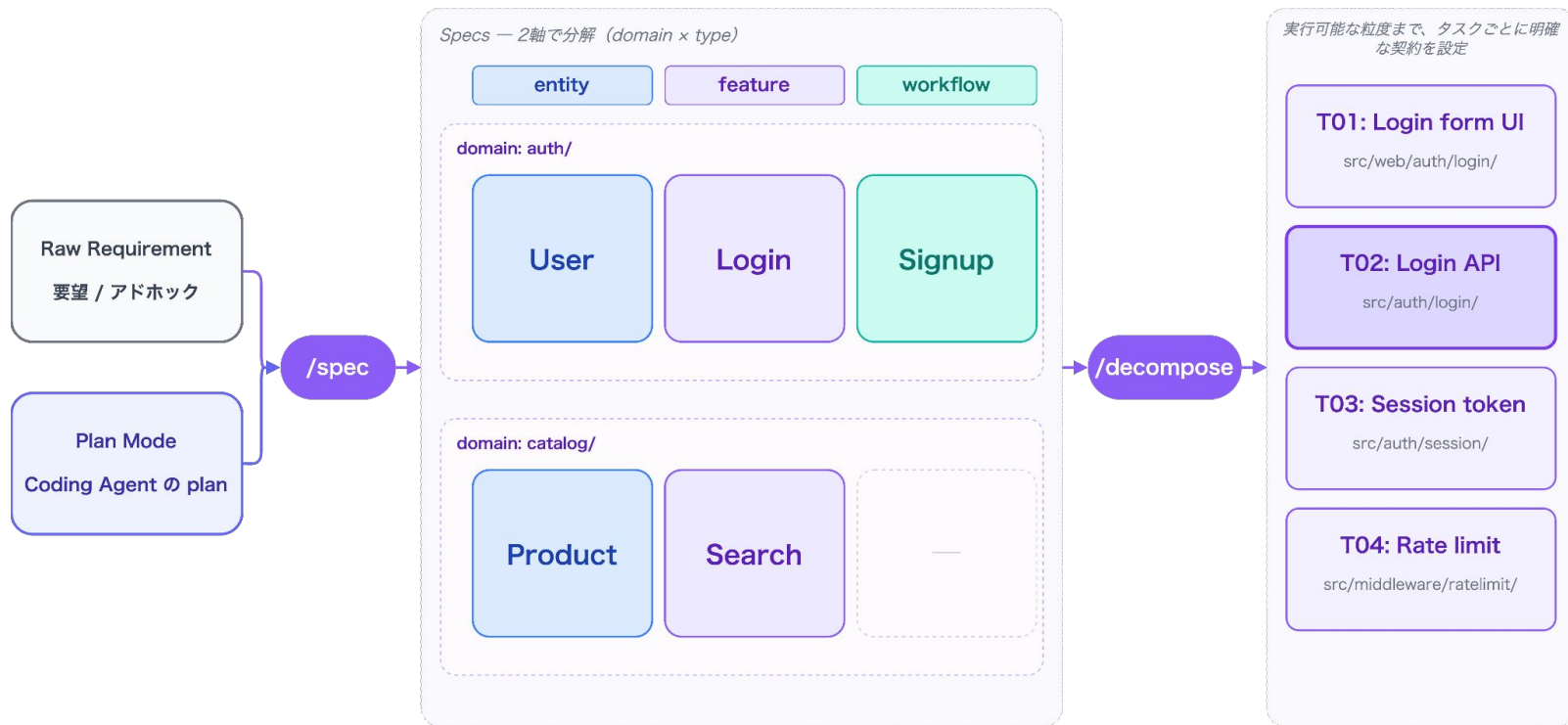
検証可能: テスト / 受入基準 / 仕様準拠 / 動いたかまでを機械が判定できる状態



証跡つき: 何をやったかが全て記録として残る

やりたいことを実行可能な粒度まで分解してドキュメント生成

作りたいものの規模やドメインによって要求/設計ドキュメントを階層的に作成し、その後それを実行可能な粒度のタスクに落とす



各タスクは単体で明確に検証可能な契約にする

- タスクのSpecに抜け漏れがないかはハーネスで機械的チェック
- checksに検証したいcommand等やspecと紐づくIDも自由に設定可能

▼ 各タスクが持つ契約 (T02 を展開)

T02: Login API

source: specs/auth/login.md#REQ-004

boundary src/auth/login/

depends_on [T01 User]

spec_covered REQ-004, REQ-005 ✓ カバレッジ検証

test_plan Red → Green → Refactor (TDD)
Given-When-Then で具体シナリオ記述

checks tests:unit tests:integration e2e:smoke
contract:grep build:typescript

boundary / depends_on / spec_covered / test_plan / checks を全タスクに定義

.alchemy > tasks > 14-auth-permission-context.md > # 認証・権限コンテキストとUIガード

```
1 ---
2 id: 14-auth-permission-context
3 title: 認証・権限コンテキストとUIガード
4 priority: high
5 depends_on:
6   - 12-frontend-routing-layout
7   - 13-api-client-tanstack
8 source: "specs/web-ui.md#REQ-017"
9 boundary:
10   - frontend/src/contexts/AuthContext.tsx
11   - frontend/src/hooks/useAuth.ts
12   - frontend/src/hooks/usePermission.ts
13   - frontend/src/components/PermissionGuard/
14   - frontend/src/components/ProtectedRoute/
15   - frontend/__tests__/
16 specs_covered:
17   - specs/web-ui.md#REQ-017
18 checks:
19   - tests:unit
20   - contract:grep="PermissionGuard|usePermission"@frontend/src/**/*.{ts,tsx}
21 ---
22
23 # 認証・権限コンテキストとUIガード
```

タスクの並列実行 + 依存関係制御をするためのDAG

できる限り早く終わって欲しいため、並列実行入れたかった！

Dependency Graph

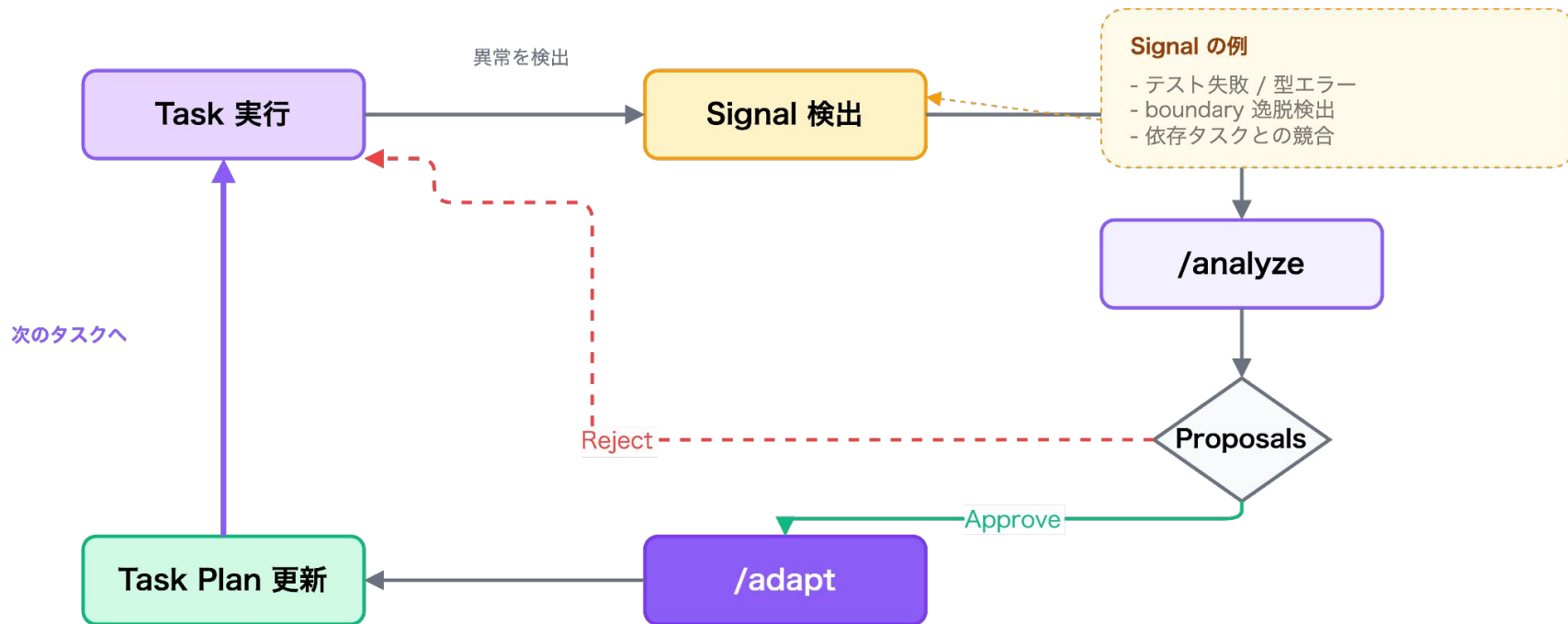


Parallel Groups

- **Group 1:** 01-project-scaffold
- **Group 2:** 02-database-schema, 11-frontend-setup
- **Group 3:** 03-user-permission-service, 12-frontend-routing-layout, 15-ui-components
- **Group 4:** 04-ticket-crud-service

全タスクを証跡を残しつつ長時間自律フィードバックループ

AI Agentが自律判断して自律改善できる要素を個別調整できる長時間自律させるための自己改善ループ & タスクごとの証跡とメモリ



最初雑に作った段階で中規模開発はできてしまった...

5 万行 / 8時間

8時間放置で5万行作成、コア体験は動いた。これが最悪の成功体験だった。

様々なソフトウェア開発に対応出来るハーネスにしたい！

色々範囲を広げてすぎて依存がどんどん増えていく... そして自由度が下がっていく

1 様々なドキュメント

文書テンプレートがたくさん！

- アーキテクチャ設計書
- データモデル / ERD / エンティティ定義
- 機能仕様書 / 画面仕様書
- API仕様書
- 業務フロー / ユースケース
- モジュール設計 / コンポーネント境界
- 非機能要件
- etc.

2 様々な検証

決定論的だけでなく推論も！

```
checks:
  - tests:unit
  - tests:integration
  - e2e:smoke
  - contract:ui=<key>
  - contract:grep=<pattern>@<glob>
  - skill:<name>
  - manual
Build / Deps / Lint (setup & integration tests)
  - build:typescript
  - build:next
  - deps:install
  - deps:prisma
  - lint:eslint
Bootstrap primitives (environment setup):
  - structure:exists=path1,path2
  - tool:exists=cmd1,cmd2
  - shell:exit0="command"
  - env:set=VAR1,VAR2
```

3 様々な技術スタック

設定ファイルだけで300行超

```
281 # # e2e:full: "npm playwright test"
282 #
283 # # =====
284 # # Node/TypeScript projects (npm)
285 # # =====
286 # # build:typescript: "npm run build"
287 # # deps:install: "npm ci"
288 # # lint:eslint: "npm run lint"
289 # # tests:unit: "npm test"
290 #
291 # # =====
292 # # Go projects
293 # # =====
294 # # build:go: "go build ./..."
295 # # tests:unit: "go test -short ./..."
296 # # tests:integration: "go test -run Integration ./..."
297 # # tests:all: "go test ./..."
298 # # lint:go: "golangci-lint run"
299 #
300 # # =====
301 # # Rust projects
302 # # =====
303 # # build:rust: "cargo build"
304 # # tests:unit: "cargo test"
305 # # tests:integration: "cargo test --test '*' "
306 # # lint:rust: "cargo clippy -- -D warnings"
307 #
```

並列実装で生じる異常

大規模開発になると、境界と仕様のドリフトが頻繁に発生することで並列実装時の編集ロックがコンフリクト

1 境界のドリフト

特に新規開発だと境界を明確に設計するのは大規模になればなるほど想定外が起こる

2 仕様のドリフト

仕様書内に書いていないものは守れない

3 依存関係のドリフト

最初のタスク分解を承認した上で進めてたが途中で発生したドリフトと自己修正でDAGの組み直しが頻繁に発生

一つ一つ修正していったが...

ハーネスが密結合に...

タスクのテンプレートを1個変えただけで、10箇所の整合性を取る必要が...

1

template / hook / verify / tests
/ policy が同時に動く. 汎用的
に作ってるから変更箇所が多い

2

1回全体をテストするのに平気で1時間とかの長時間かかる.

3

修正してもどこを修正したからうまくいった/失敗したのかを評価不能に

そしてエージェントハーネスは肥大化

Script: 25,000行・Markdown: 10,000行・Hooks: 1,500行

**ハーネスでなんでもやろうとすると、
エージェントを制御するための仕組み
自体が制御不能になる**

—

そしてハーネスは解体された

まとめ: 今だったらこう分ける！

1 Planning Agent

要求・計画からタスク契約に落とす

2 Execution Agent

タスク契約を読む → 境界内でTDD実装する

3 Verification Agent

タスク契約を読む → checks と evidence で完了判定する

4 Adaptation Agent

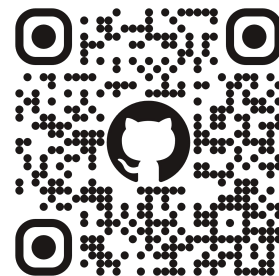
failure / evidence を読む → 次の提案を出す

目的ごとに小さく分けてシンプルかつ変更容易性の高いハーネスを作ろう！

ありがとうございました！

質問やお問い合わせ等あればお気軽に連絡ください！

X: @gota_bara



cc-sddのGithub